QRV — QRVoo: Gerador de QR Code para Passagens Aéreas

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.2

Campo	Descrição
Projeto	QRV — QRVoo: Gerador de QR Code para Passagens Aéreas
Gerente de Projeto	João Ramos
Gerente/Responsável de GCS/DevOps	João Ramos
Par de revisão	josesilveiraa07 (colaborador com acesso de escrita)
Fábrica de Software / Squad	Individual — João Ramos (Engenharia de Software, Católica SC)
Repositório principal	https://github.com/joaoramosss/qrvoo
Localização deste plano	`docs/PGCS_QRVoo.pdf` (item de configuração versionado)

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
22/09/2026	1.1	Adaptação do template ao projeto QRVoo: ferramentas, pipelines, ambientes, convenções e marcos.	João Ramos
28/10/2026	1.2	Inclusão dos environments staging e production (revisor obrigatório), ruleset da main sem bypass, par de revisão com acesso de escrita, publicação do PGCS e das evidências no repositório, Python 3.14 e release v1.1.0.	João Ramos

Sumário

1. Introdução . . . 3
1.1 Finalidade . . . 3
1.2 Escopo . . . 3
1.3 Definições, Acrônimos e Abreviações . . . 3
1.4 Referências . . . 4
1.5 Visão Geral . . . 4
2. Cultura DevOps e Papéis na Gerência de Configuração . . . 4
3. Ferramentas, Ambientes e Infraestrutura . . . 5
3.1 Ferramentas do Pipeline . . . 5
3.2 Ambientes e Infraestrutura . . . 5
3.3 Estratégia de Branching e Versionamento . . . 6
4. Identificação da Configuração . . . 6
4.1 Itens de Configuração . . . 6
4.2 Convenção de Nomenclatura . . . 7
4.3 Estrutura do Repositório . . . 7
5. Integração Contínua (CI) . . . 7
5.1 Etapas do Pipeline de CI . . . 7
5.2 Política de Pull Request e Revisão de Código . . . 7
5.3 Critérios de Qualidade (Quality Gate) . . . 8
6. Entrega e Implantação Contínua (CD) . . . 8
6.1 Etapas do Pipeline de CD . . . 8
6.2 Estratégias de Implantação . . . 8
6.3 Rollback e Recuperação . . . 9
7. Infraestrutura como Código (IaC) . . . 9
7.1 Versionamento de Infraestrutura . . . 9
7.2 Ambientes Efêmeros . . . 9
8. Controle de Configuração e Mudança . . . 9
8.1 Processo de Solicitação de Mudança . . . 9
8.2 Comitê de Controle de Mudança (CCM) . . . 9
9. Baselines e Releases . . . 10
9.1 Geração Automática de Baseline . . . 10
9.2 Notas de Versão (Release Notes) . . . 10
10. Estimativa do Status de Configuração . . . 10
10.1 Monitoramento e Observabilidade . . . 10
10.2 Métricas DORA . . . 10
10.3 Relatórios e Auditorias . . . 10
11. Marcos . . . 11
12. Treinamento e Recursos . . . 11
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros . . . 11
14. Quadro-Resumo do Pipeline CI/CD . . . 12

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) do projeto QRVoo ao longo de todo o seu ciclo de vida, com ênfase em práticas DevOps: Integração Contínua (CI), Entrega Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. O objetivo é identificar os itens de configuração, manter sua integridade e controlar as mudanças com o mínimo de burocracia e o máximo de automação.

O QRVoo é uma aplicação em Python que gera QR Codes que, ao serem lidos pelo celular, abrem diretamente a busca de passagens aéreas em sites como Skyscanner, Kayak, Momondo e Google Flights, já preenchida com origem, destino, datas e número de passageiros. A aplicação oferece uma interface de linha de comando e uma interface web, e é distribuída como imagem Docker.

1.1 Finalidade

Estabelecer o padrão a ser seguido no desenvolvimento do QRVoo, garantindo rastreabilidade, versionamento e controle de mudanças, apoiado por pipelines automatizados de build, teste, análise de segurança, empacotamento e publicação.

1.2 Escopo

Este PGC é destinado ao desenvolvedor João Ramos, responsável pelo desenvolvimento e pela operação do sistema QRVoo, e ao par de revisão josesilveiraa07, como atividade da disciplina de Gerência de Configuração do curso de Engenharia de Software da Católica de Santa Catarina. Abrange o código-fonte da aplicação (módulos de montagem de links, geração de QR Code, CLI e interface web), os testes automatizados, os pipelines do GitHub Actions, o Dockerfile, o arquivo de composição do ambiente de homologação, a configuração dos environments e da proteção da branch main, a documentação técnica e este próprio plano, todos mantidos no repositório principal.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que se integra na branch principal com alta frequência
Ruleset	Conjunto de regras do GitHub que protege branches (PR obrigatório, checks, bloqueio de force push)
Environment	Ambiente de implantação do GitHub Actions com regras de proteção próprias (ex.: revisor obrigatório)
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH), ex.: 1.1.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
IATA	Código de 3 letras que identifica aeroportos (ex.: GRU, LIS, JOI)
GHCR	GitHub Container Registry, registro de imagens de container do GitHub
SAST / SCA	Análise estática de segurança do código / Análise de componentes de terceiros

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Aula 1 publicada no repositório GitHub do projeto ongconecta (proteção de branch, par de revisão e environments).
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- README.md do repositório QRVoo (instruções de execução e contribuição).
- Registros de decisão de arquitetura: docs/adr/0001-python-qrcode-flask.md e docs/adr/0002-trunk-based-ghcr.md.
- Documentação da biblioteca qrcode (PyPI), do Flask e do GitHub Actions.
- Conventional Commits 1.0.0 e Semantic Versioning 2.0.0.

1.5 Visão Geral

Diferente de processos tradicionais, em que a GCS é manual e concentrada em marcos formais, neste plano ela faz parte do pipeline de entrega. Cada commit integrado, revisado, testado e publicado automaticamente gera evidências de rastreabilidade: logs de execução no GitHub Actions, aprovações nos Pull Requests, histórico de deployments por environment, tags de versão, imagens publicadas no GHCR e releases com notas geradas automaticamente.

2. Cultura DevOps e Papéis na Gerência de Configuração

O projeto tem um desenvolvedor principal, que exerce a maior parte dos papéis seguindo o princípio "você constrói, você executa", e um par de revisão com acesso de escrita ao repositório. A automação do pipeline e as regras do GitHub garantem que nenhuma verificação dependa apenas de disciplina manual.

Papel	Responsável	Responsabilidades no QRVoo
Gerente de Projeto / Product Owner	João Ramos	Prioriza as issues do GitHub, decide quais sites de viagem suportar e acompanha as métricas DORA.
Engenheiro DevOps / Release Engineer	João Ramos	Mantém os workflows ci.yml e release.yml, o ruleset da main, os environments, as tags SemVer e as imagens no GHCR.
Desenvolvedor(a)	João Ramos	Implementa as funcionalidades em branches curtas, escreve testes com pytest e abre Pull Requests.
Par de revisão	josesilveiraa07	Colaborador com acesso de escrita (Write). Revisa e aprova os Pull Requests; sem a aprovação dele nenhum PR é integrado na main.
Aprovador de produção	João Ramos	Revisor obrigatório do environment production: aprova cada implantação disparada por tag vX.Y.Z.
SRE / Operações	João Ramos	Monitora o endpoint /health, executa rollback para a tag anterior e registra incidentes em issues.
QA / Automação de Testes	João Ramos + pipeline	Mantém a suíte de testes unitários e de integração (Flask test client) e o smoke test do container.
Segurança (DevSecOps)	João Ramos + pipeline	Mantém bandit (SAST), pip-audit e Dependabot (SCA). A aplicação não usa segredos próprios.
CCM	Professor orientador (consultivo) e João Ramos	Acionado apenas para mudanças de alto impacto, conforme a seção 8.2.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta adotada	Finalidade
Controle de versão	Git + GitHub	Versionamento de código, pipelines, Dockerfile, compose e documentação
Proteção de branch	GitHub Rulesets	PR obrigatório com 1 aprovação, checks obrigatórios, sem bypass
Orquestração de CI/CD	GitHub Actions	Build, lint, testes, análise de segurança, imagem e release
Ambientes de implantação	GitHub Environments (staging, production)	Rastreio de deployments e gate de aprovação em produção
Linguagem e bibliotecas	Python 3.14, qrcode[pil], Flask, Gunicorn	Geração do QR Code e interface web
Testes e cobertura	pytest + pytest-cov	Testes unitários e de integração com cobertura mínima
Lint	Ruff	Padrões de código e ordenação de imports
Empacotamento	Docker	Imagem versionada e imutável da aplicação
Registro de artefatos	GitHub Container Registry (GHCR)	Armazenamento das imagens por SHA e por versão
Infraestrutura como Código	Dockerfile + Docker Compose (infra/)	Definição versionada do ambiente de execução
Gestão de segredos	GitHub Secrets (GITHUB_TOKEN)	Credencial temporária para publicar no GHCR
Monitoramento	Endpoint /health + HEALTHCHECK do Docker	Verificação de saúde do serviço
SAST / SCA	Bandit / pip-audit + Dependabot	Vulnerabilidades no código e nas dependências
Rastreamento de tarefas	GitHub Issues	Vinculação de commits e PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Os ambientes são definidos por arquivos versionados (Dockerfile e infra/docker-compose.yml) e registrados como GitHub Environments, o que garante paridade entre eles: a mesma imagem testada no CI é a que vai para staging e é promovida para produção.

Ambiente	Finalidade	Gatilho de promoção / proteção
Desenvolvimento	Máquina do desenvolvedor com ambiente virtual Python (venv) e servidor Flask local (porta 8000).	Commit/push em branch feature/*
Homologação — environment **staging**	Job "Deploy em staging" do ci.yml: carrega a imagem testada, publica no GHCR com as tags <sha> e latest e executa smoke test da imagem publicada. Pode ser executada localmente com infra/docker-compose.yml (tag latest).	Merge na main com CI verde. Deploy restrito à branch main.
Produção — environment **production**	Workflow release.yml: promove a imagem <sha> para ghcr.io/joaoramosss/qrvoo:X.Y.Z, executa smoke test e cria a GitHub Release.	Tag vX.Y.Z + aprovação obrigatória do revisor João Ramos (joaoramosss), sem bypass de administrador.

Topologia: um único serviço web stateless (Gunicorn com 2 workers) exposto na porta 8000, sem banco de dados. Não há credenciais de aplicação; a única credencial usada é o GITHUB_TOKEN, gerado automaticamente pelo GitHub a cada execução do pipeline.

3.3 Estratégia de Branching e Versionamento

O projeto adota Trunk-Based Development (GitHub Flow), com branches de vida curta:

- **main** — sempre implantável. Protegida pelo ruleset "main" (status Active, bypass list vazia, inclusive para administradores), que exige: alterações somente por Pull Request com 1 aprovação; status checks "Lint, testes e segurança" e "Build e smoke test da imagem" aprovados; bloqueio de force push e de exclusão da branch. Push direto é recusado pelo GitHub (evidência em docs/evidencias/push-main-recusado.png).
- **feature/<issue>-<descrição-curta>** — uma branch por mudança, com duração ideal de até 2 dias.
- **hotfix/<descrição>** — correções urgentes, abertas a partir da main e integradas por PR com a mesma aprovação.
- **release/x.y** — não utilizada; a estabilização é feita por testes automatizados antes da tag.

Versionamento: Versionamento Semântico (MAJOR.MINOR.PATCH) com Conventional Commits (feat, fix, chore, docs, refactor, test). A versão atual (1.1.0) está em pyproject.toml e em src/qrvoo/__init__.py e é exibida pelo endpoint /health; as notas de release são geradas automaticamente pelo GitHub a partir dos PRs e commits desde a última tag.

4. Identificação da Configuração

4.1 Itens de Configuração

Todos os itens abaixo são versionados no repositório principal:

Item de configuração	Localização
Código-fonte da aplicação	src/qrvoo/ (links.py, qr.py, cli.py, web.py, templates/)
Testes automatizados	tests/ (test_links.py, test_qr.py, test_cli.py, test_web.py)
Pipelines como código	.github/workflows/ci.yml e .github/workflows/release.yml
Dockerfile e .dockerignore	Raiz do repositório
Infraestrutura como código	infra/docker-compose.yml
Dependências travadas	requirements.txt e requirements-dev.txt (versões fixas)
Configuração do projeto	pyproject.toml (metadados, pytest, cobertura, ruff)
Atualização de dependências	.github/dependabot.yml
Modelo de Pull Request	.github/pull_request_template.md
Plano de Gerenciamento de Configuração	docs/PGCS_QRVoo.pdf (este documento)
Evidências de GCS	docs/evidencias/ (push recusado, ruleset, colaborador, environments)
Documentação técnica	README.md, CHANGELOG.md, LICENSE, docs/adr/
Regras do repositório (ruleset, environments, colaboradores)	Configuradas no GitHub (Settings); estado registrado por prints em docs/evidencias/
Scripts de migração de banco	N/A — a aplicação não possui banco de dados
Segredos	Não versionados; somente GitHub Secrets (GITHUB_TOKEN automático)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição-curta>	feature/12-suporte-kiwi
Branch de hotfix	hotfix/<descrição>	hotfix/url-skyscanner
Commit	<tipo>(<escopo>): <descrição>	feat(links): adiciona suporte ao Kiwi.com
Tag de release	vX.Y.Z	v1.1.0
Imagem (build)	ghcr.io/<usuário>/qrvoo:<sha>	ghcr.io/joaoramosss/qrvoo:3f2a9c1…
Imagem (release)	ghcr.io/<usuário>/qrvoo:<X.Y.Z>	ghcr.io/joaoramosss/qrvoo:1.1.0
Arquivo de teste	tests/test_<módulo>.py	tests/test_links.py
ADR	docs/adr/NNNN-<assunto>.md	docs/adr/0001-python-qrcode-flask.md
Evidência	docs/evidencias/<assunto>.png	docs/evidencias/push-main-recusado.png
Environment	nome em inglês, minúsculo	staging, production

4.3 Estrutura do Repositório

Diretório / arquivo	Conteúdo
/src/qrvoo	Pacote Python da aplicação e template HTML
/tests	Testes automatizados (pytest)
/.github/workflows	Pipelines de CI + staging (ci.yml) e de release/produção (release.yml)
/.github	Dependabot e modelo de Pull Request
/infra	docker-compose.yml do ambiente de homologação/produção
/docs	PGCS_QRVoo.pdf (este plano)
/docs/adr	Registros de decisões de arquitetura
/docs/evidencias	Prints de verificação da GCS e índice (README.md)
/ (raiz)	Dockerfile, pyproject.toml, requirements*.txt, README, CHANGELOG, LICENSE

5. Integração Contínua (CI)

Todo Pull Request para a main e todo push na main disparam automaticamente o workflow CI (.github/workflows/ci.yml). Nenhuma mudança é integrada sem que o pipeline termine com sucesso e sem a aprovação do par de revisão.

5.1 Etapas do Pipeline de CI

1 Checkout do código e instalação das dependências com cache do pip (actions/setup-python, Python 3.14).
2 Instalação do pacote em modo editável (build da aplicação).
3 Análise estática de código com Ruff (erros, imports, boas práticas).
4 Testes unitários com medição de cobertura (pytest-cov, mínimo de 80%).
5 Testes de integração da interface web e da CLI (Flask test client e execução da CLI).
6 Análise de segurança do código com Bandit (SAST) e das dependências com pip-audit (SCA).
7 Build da imagem Docker e smoke test do endpoint /health (job "Build e smoke test da imagem"). Em push na main, a imagem é salva como artefato para ser reutilizada pelo staging, sem novo build.

5.2 Política de Pull Request e Revisão de Código

- Todo PR deve referenciar uma issue e seguir o modelo .github/pull_request_template.md.

- A regra de "quatro olhos" é aplicada: todo PR precisa de 1 aprovação do par de revisão josesilveiraa07, colaborador com acesso de escrita (Write). A exigência é imposta pelo ruleset da main.
- PRs devem ser pequenos e de vida curta; o checklist do template inclui "Aprovado pelo par de revisão".
- O merge só é permitido com os checks obrigatórios verdes e a aprovação registrada. Push direto na main é recusado, inclusive para o administrador.

5.3 Critérios de Qualidade (Quality Gate)

Critério	Limite	Ferramenta
Cobertura de testes	≥ 80% (bloqueante)	pytest --cov-fail-under=80
Testes	100% aprovados	pytest
Lint	Zero erros	ruff check .
Segurança do código	Zero achados de severidade média ou alta	bandit -r src -ll
Dependências	Zero vulnerabilidades conhecidas	pip-audit
Imagem	Endpoint /health responde em até 30 s	Smoke test no CI
Revisão	1 aprovação do par de revisão	Ruleset da main
Integração	Somente via Pull Request; sem bypass	Ruleset da main

Na versão 1.0.0, a suíte possuía 26 testes com cobertura de 98%.

6. Entrega e Implantação Contínua (CD)

A mesma imagem construída e testada no CI é publicada em staging e depois promovida para produção, sem recompilação, identificada pelo SHA do commit.

6.1 Etapas do Pipeline de CD

8 Após o merge na main, o job "Deploy em staging" (environment **staging**, restrito à main) carrega a imagem salva pelo CI.

9 Publicação da imagem no GHCR com as tags <sha> e latest.

10 Smoke test da imagem baixada do GHCR (validação do staging); o deployment fica registrado no environment.

11 Criação da tag vX.Y.Z pelo desenvolvedor, disparando o workflow Release.

12 O job aguarda a aprovação do revisor obrigatório do environment **production** (João Ramos), sem bypass de administrador.

13 Promoção da imagem <sha> para a tag de versão X.Y.Z no GHCR, sem novo build.

14 Smoke test da versão promovida (health check).

15 Criação automática da GitHub Release com notas de versão geradas (baseline).

6.2 Estratégias de Implantação

Estratégia	Descrição	Uso no QRVoo
Rolling update	Substituição gradual das instâncias antigas pelas novas.	Adotada: docker compose pull && up -d troca o container pela nova versão.
Blue-Green	Duas versões coexistem e o tráfego é comutado de uma vez.	Não adotada (serviço único e sem usuários concorrentes).
Canary Release	Nova versão recebe fração do tráfego real.	Não adotada nesta fase.
Feature Flags	Funcionalidades implantadas desligadas.	Não adotada; novos sites entram só após testes.

6.3 Rollback e Recuperação

Toda implantação é reversível: as imagens anteriores permanecem no GHCR com suas tags de versão. Em caso de falha, basta definir QRVOO_VERSION com a versão anterior e executar `docker compose -f infra/docker-compose.yml up -d`. No código, a reversão é feita com `git revert` via PR (com aprovação do par), seguido de nova tag PATCH.

7. Infraestrutura como Código (IaC)

O ambiente de execução do QRVoo é descrito por código versionado: o Dockerfile define a imagem (Python 3.14 slim, usuário sem privilégios, HEALTHCHECK) e o infra/docker-compose.yml define como o serviço é executado. Ambos passam pelo mesmo fluxo de PR, revisão e pipeline da aplicação.

7.1 Versionamento de Infraestrutura

- Alterações no Dockerfile, no compose ou nos workflows seguem PR + aprovação do par + pipeline; o CI reconstrói a imagem e executa o smoke test antes do merge.
- Não há state file, pois não são provisionados recursos em nuvem; o estado desejado está integralmente no repositório.
- As configurações feitas pela interface do GitHub (ruleset, environments e colaboradores) não ficam em arquivo; seu estado é registrado em docs/evidencias/ e conferido na auditoria.

7.2 Ambientes Efêmeros

Cada execução do CI cria um container efêmero a partir da imagem recém-construída, valida o /health e o descarta ao final do job, funcionando como ambiente de revisão isolado por Pull Request.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina seguem o fluxo: issue no GitHub → branch feature/* → commits Conventional → Pull Request → checklist → pipeline de CI → aprovação do par de revisão → merge na main → deploy em staging → tag de release → aprovação em production. Esse fluxo constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Composição: João Ramos (responsável) e o professor da disciplina, em caráter consultivo. O CCM é acionado somente nos casos abaixo; nos demais, o quality gate automatizado e a aprovação do par substituem o comitê.

- Mudança de versão MAJOR (quebra de compatibilidade da CLI ou do endpoint /qr.png).
- Troca de linguagem, framework ou registro de imagens.
- Hotfix aplicado fora do fluxo padrão, alteração do ruleset da main ou dos environments, ou desativação de alguma etapa do quality gate.

A decisão é registrada em issue com o rótulo "ccm" e, quando for de arquitetura, em um novo ADR.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada release bem-sucedida gera uma baseline: a tag SemVer associada ao commit, a imagem ghcr.io/joaoramosss/qrvoo:X.Y.Z, o registro do deployment no environment production e o estado do Dockerfile, do compose e deste plano naquele commit. A baseline inicial é a v1.0.0; seguem-se a v1.0.1 (atualizações de dependências via Dependabot) e a v1.1.0 (environments, ruleset, par de revisão e publicação do PGCS).

9.2 Notas de Versão (Release Notes)

As notas são geradas automaticamente pelo comando `gh release create --generate-notes` a partir dos PRs e commits desde a última tag. O CHANGELOG.md registra o resumo de cada versão.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

A aplicação expõe o endpoint /health, que retorna status e versão em execução. O HEALTHCHECK do Docker o consulta a cada 30 segundos, e o Gunicorn registra os acessos e erros na saída padrão (docker logs). Falhas são registradas como issues com o rótulo "incidente". O histórico de deployments de cada environment fica disponível na aba Environments do repositório.

10.2 Métricas DORA

Métrica	Descrição	Como é medida no QRVoo
Deployment Frequency	Frequência de implantação em produção.	Número de deployments no environment production (GitHub Releases) por mês.
Lead Time for Changes	Tempo entre o commit e a produção.	Data do primeiro commit do PR até a aprovação do deployment em production.
Change Failure Rate	Percentual de implantações com falha.	Releases seguidas de hotfix ou rollback / total de releases.
MTTR	Tempo médio para restaurar o serviço.	Abertura até fechamento das issues "incidente".

10.3 Relatórios e Auditorias

O histórico da aba Actions (builds, testes, cobertura, imagens), o histórico de commits e PRs com suas aprovações, os deployments dos environments, as tags e as Releases constituem o insumo de auditoria de GCS. Na verificação em laboratório, a auditoria consiste em conferir:

- repositório público e acessível, com a estrutura da seção 4.3 e este plano em docs/PGCS_QRVoo.pdf;
- recusa do push direto na main (docs/evidencias/push-main-recusado.png e ruleset "main" ativo, sem bypass);
- par de revisão josesilveiraa07 com acesso de escrita e aprovação registrada nos PRs;
- environments staging e production, este com João Ramos como revisor obrigatório;
- execução verde do CI e do deploy em staging na main, tags v1.0.0, v1.0.1 e v1.1.0, imagens publicadas no GHCR e correspondência entre os itens da seção 4.1 e o repositório.

11. Marcos

Marco	Descrição	Data prevista
M1	Criação do repositório, estrutura inicial e proteção da branch main	22/09/2026
M2	Pipeline de CI com lint, testes, cobertura, SAST e SCA funcionando	23/09/2026
M3	Publicação automática da imagem no GHCR	23/09/2026
M4	Primeira release automatizada (v1.0.0) com notas geradas	24/09/2026
M5	Ruleset da main sem bypass, par de revisão, environments staging e production, PGCS e evidências publicados no repositório	28/10/2026
M6	Release v1.1.0 aprovada no environment production	28/10/2026
M7	Verificação do PGCS e do repositório em laboratório	Conforme ordem de publicação no Teams

12. Treinamento e Recursos

Pessoal: um desenvolvedor (João Ramos), exercendo os papéis descritos na seção 2, e um par de revisão (josesilveiraa07) com acesso de escrita.

Recursos: computador com Python 3.14, Git e Docker Desktop; conta gratuita no GitHub com Actions, Environments e GHCR habilitados (repositório público). Não há custo de infraestrutura.

Treinamento: estudo da documentação oficial do GitHub Actions, Rulesets, Environments, Docker e da biblioteca qrcode; conteúdo da disciplina sobre GCS (incluindo a aula 1 do projeto ongconecta); prática com Conventional Commits e SemVer durante o próprio desenvolvimento do projeto.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Não há software desenvolvido por subcontratados. As bibliotecas de terceiros são itens de configuração, com versões fixadas em requirements.txt (qrcode 8.2, Pillow 13.0.0, Flask 3.1.3, Gunicorn 23.0.0) e requirements-dev.txt (pytest 9.1.1, pytest-cov 7.1.0, ruff 0.16.8, bandit 1.9.4, pip-audit 2.10.1).

- Análise automática de vulnerabilidades com pip-audit em todo PR.
- Dependabot propõe atualizações semanais de pacotes pip e mensais de actions e da imagem base Docker, submetidas ao mesmo fluxo de PR, revisão e pipeline.
- Os sites de viagem (Skyscanner, Kayak, Momondo, Google Flights) são serviços externos acessados apenas por URL pública; mudanças no formato de URL são tratadas como correção (fix) em links.py, com teste correspondente.

14. Quadro-Resumo do Pipeline CI/CD

Etapa	Descrição	Gatilho
CI	Ruff, pytest com cobertura ≥ 80%, Bandit, pip-audit, build da imagem e smoke test.	Abertura/atualização de Pull Request
Revisão	Aprovação do par josesilveiraa07 (acesso Write).	PR com CI verde
Merge	Integração da mudança na main (push direto bloqueado pelo ruleset).	CI verde + 1 aprovação
CD — Staging	Environment staging: publicação da imagem <sha>/latest no GHCR e smoke test.	Merge na main
Gate de Release	Environment production: aprovação obrigatória de João Ramos.	Criação da tag vX.Y.Z
CD — Produção	Promoção da imagem para X.Y.Z, smoke test e GitHub Release com notas (baseline).	Aprovação do gate de release

Este plano mantém os quatro pilares da GCS (identificação, controle de versão, controle de mudanças e auditoria/status), todos suportados por automação no GitHub Actions e pelas regras de proteção do repositório.